



**UNIVERSIDADE FEDERAL DA FRONTEIRA SUL
CAMPUS DE CHAPECÓ
CURSO DE CIÊNCIA DA COMPUTAÇÃO**

NATANAEL HENRIK ZAGO

**USANDO ESQUEMA GraphQL PARA GERAÇÃO DE CONSULTAS DE FORMA
ALEATÓRIA**

**CHAPECÓ
2023**

NATANAEL HENRIK ZAGO

**USANDO ESQUEMA GRAPHQL PARA GERAÇÃO DE CONSULTAS DE FORMA
ALEATÓRIA**

Trabalho de conclusão de curso apresentado como requisito para obtenção do grau de Bacharel em Ciência da Computação da Universidade Federal da Fronteira Sul.
Orientador: Prof. Dr. Samuel da Silva Feitosa

CHAPECÓ
2023

Zago, Natanael Henrik

Usando Esquema GraphQL Para Geração De Consultas De Forma Aleatória / Natanael Henrik Zago. – 2023.

33 f.: il.

Orientador: Prof. Dr. Samuel da Silva Feitosa.

Trabalho de conclusão de curso (graduação) – Universidade Federal da Fronteira Sul, curso de Ciência da Computação, Chapecó, SC, 2023.

1. Geração de Código Aleatório. 2. GraphQL. 3. Testes de API.
I. Feitosa, Prof. Dr. Samuel da Silva, orientador. II. Universidade Federal da Fronteira Sul. III. Título.

NATANAEL HENRIK ZAGO

**USANDO ESQUEMA GraphQL PARA GERAÇÃO DE CONSULTAS DE FORMA
ALEATÓRIA**

Trabalho de conclusão de curso apresentado como requisito para obtenção do grau de Bacharel em Ciência da Computação da Universidade Federal da Fronteira Sul.

Orientador: Prof. Dr. Samuel da Silva Feitosa

Este trabalho de conclusão de curso foi defendido e aprovado pela banca avaliadora em: 5/9/2023.

BANCA AVALIADORA

Prof. Dr. Samuel da Silva Feitosa – UFFS

Prof. Dr. Andrei De Almeida Sampaio Braga –
UFFS

Prof. Dr. Bráulio Adriano de Mello – UFFS

RESUMO

O GraphQL é uma linguagem de consulta de API desenvolvida pelo Facebook que é atualmente *open source*. Sua principal característica está em disponibilizar somente as informações solicitadas para um cliente de API, desse modo ela gera economia de recursos de rede e maior flexibilidade na comunicação entre cliente e API. Entretanto, poucos são os utilitários para automatizar testes de APIs GraphQL. Um ponto importante para se gerar casos de testes é a criação de consultas válidas, porém, esta tarefa pode consumir bastante tempo dos desenvolvedores, além de não atingir todos os casos de testes possíveis. Visando ajudar os desenvolvedores de APIs GraphQL, o presente trabalho tem como intuito a geração de consultas de forma automática baseadas no esquema da API GraphQL, gerando consultas válidas para a realização de testes, evitando trabalho humano dispensável e produzindo ampla quantidade de consultas. Como ferramenta para geração das consultas baseadas em esquema, será utilizada a linguagem de programação funcional Haskell, bem como testes baseados em propriedades.

Palavras-chave: Geração de Código Aleatório. GraphQL. Testes de API.

ABSTRACT

GraphQL is an API query language developed by Facebook that is currently open source. Its main feature is to make only the requested information available to an API client, thus saving network resources and providing greater flexibility in the communication between the client and the API. However, there are few utilities to automate testing of GraphQL APIs. An important point for generating test cases is the creation of valid queries, however, this task can consume a lot of time for developers, in addition to not reaching all possible test cases. Aiming to help developers of GraphQL APIs, this work aims to automatically generate queries based on the GraphQL API schema, generating valid queries for testing, avoiding unnecessary human work and producing a large number of queries. As a tool for generating schema-based queries, the Haskell functional programming language will be used, as well as property-based tests.

Keywords: Random Code Generation. GraphQL. Test API.

LISTA DE ILUSTRAÇÕES

Figura 1 – Exemplo de definição de tipo <i>GraphQL</i>	20
Figura 2 – Exemplo de consulta <i>GraphQL</i>	20
Figura 3 – Exemplo de retorno no formato JSON <i>GraphQL</i>	20

LISTA DE TABELAS

Tabela 1 – Cronograma previsto para as atividades propostas.	29
--	----

SUMÁRIO

1	INTRODUÇÃO	15
1.1	Apresentação	15
1.2	Problema de Pesquisa	15
1.3	Hipótese de Pesquisa	16
1.4	Objetivos	16
1.4.1	Objetivos gerais	16
1.4.2	Objetivos específicos	16
1.5	Justificativa	16
1.6	Organização do Texto	17
2	REVISÃO BIBLIOGRÁFICA	19
2.1	APIs WEB	19
2.2	GraphQL	19
2.3	Teste Baseado em Propriedades	20
2.4	Geração de Código Aleatório	21
3	TRABALHOS RELACIONADOS	23
3.1	Teste de desvio: uma geração de casos de teste - Técnica para APIs GraphQL	23
3.2	AutoGraphQL: Uma ferramenta de geração de teste automatizada para GraphQL	23
3.3	Teste automático baseado em propriedade de APIs GraphQL	24
3.4	Fuzzing de caixa branca e caixa preta para APIs GraphQL	24
3.5	QuickREST: Geração de teste baseada em propriedade de APIs RESTful descritas pela OpenAPI	25
4	METODOLOGIA	27
5	CRONOGRAMA	29
6	CONSIDERAÇÕES FINAIS	31
	REFERÊNCIAS	33

1 INTRODUÇÃO

1.1 APRESENTAÇÃO

No decorrer dos últimos anos têm se notado um aumento contínuo no uso de aplicações que interagem com APIs web, estas vão desde aplicações web e desktop, até sistemas instalados em dispositivos móveis, que necessitam buscar algum recurso conforme iteração do usuário ou sistema, por exemplo buscar informações atualizadas do clima ou receber e enviar mensagens. Para tornar possível essa transmissão se faz necessário o uso de tecnologias específicas para tal comunicação (2). Um dos padrões mais utilizados para construção de APIs é o REST (Representational State Transfer), que apesar de seus inúmeros benefícios, possui muitas vezes algumas limitações, como por exemplo a transferência de dados desnecessários para o cliente, assim utilizando recursos da rede e processamento dispensáveis. Com o intuito de otimizar a transferência de dados entre a comunicação dos seus serviços em 2012 o Facebook lançou o GraphQL (Graph Query Language), que é uma linguagem de consulta para comunicação entre cliente e API (3).

O GraphQL tem como grande vantagem que somente o que é solicitado pelo cliente vai ser disponibilizado, e nada mais, assim gerando uma economia no uso de rede e processamento de dados. Apesar de ter sido desenvolvido pelo Facebook, em 2015 o GraphQL se tornou open source e passou a ser administrado pela GraphQL Foundation, o que incentivou sua adesão pela comunidade de desenvolvedores, sendo grande o número de empresas que passou adotá-lo na comunicação de suas APIs, um exemplo disso é o Github(3).

Contudo, por se tratar de uma tecnologia relativamente nova, existem poucas pesquisas para automatização de geração de consultas para APIs GraphQL, o que poderia vir a corroborar na área de testes, e assim contribuir na construção de APIs mais robustas e seguras, com menor suscetibilidade a falhas. Porém, para tornar a geração de testes automáticos viáveis e com maior cobertura de casos uma abordagem é a geração autônoma de consultas GraphQL válidas para a API, assim evitando trabalho humano para criação das consultas de teste. Neste ponto a geração de consultas aleatórias em GraphQL viria a ser um utilitário para possibilitar a execução de testes de forma mais independente e com grande cobertura de casos (8).

Este trabalho propõe-se para a geração de consultas GraphQL de forma aleatória baseados no esquema da API GraphQL, repetindo suas regras e gerando assim consultas válidas, deste modo oferecendo um aparato a mais para facilitar a construção de testes para APIs que envolvem a tecnologia GraphQL.

1.2 PROBLEMA DE PESQUISA

É possível criar uma ferramenta para geração de consultas GraphQL que sejam válidas e de forma aleatória, estas baseadas no esquema da API para geração de casos de testes utilizando

a linguagem Haskell?

1.3 HIPÓTESE DE PESQUISA

A geração de consultas válidas de forma aleatória para APIs GraphQL é útil para efetuar testes de API com maior cobertura de casos, sem necessidade de esforço humano para gerar as consultas.

1.4 OBJETIVOS

1.4.1 Objetivos gerais

O objetivo geral deste trabalho é criar uma ferramenta para realizar a geração de consultas GraphQL de forma aleatória baseados no esquema da API GraphQL.

1.4.2 Objetivos específicos

- Revisar na literatura os trabalhos propostos para geração de consultas GraphQL de forma aleatória;
- Estudar o formato e a especificação da linguagem GraphQL;
- Criar ferramenta para geração de consultas GraphQL de forma aleatória utilizando a linguagem de programação Haskell;
- Implementar testes baseados em propriedades utilizando os casos de teste gerados como entrada.
- Testar as consultas geradas automaticamente em uma API GraphQL de teste.

1.5 JUSTIFICATIVA

Desenvolver software robusto e confiável é de extrema importância em um mundo onde cada vez mais a tecnologia está presente no cotidiano das pessoas e empresas. Desde a área de saúde e transporte e até mesmo nas atividades de lazer e entretenimento existe o uso de softwares pra aprimorar e auxiliar. Algo muito importante no desenvolvimento de software é a parte de testes, onde se objetiva simular possíveis cenários de uso, visando antecipar o encontro de erros antes que chegue ao usuário final. Portanto ter maneiras fáceis de criar testes vem auxiliar essa área tão importante para a qualidade de software. Uma tecnologia presente atualmente no segmento de software é o GraphQL que é uma ferramenta para comunicação entre cliente e API (Interface de Programação de Aplicação). O presente trabalho tem como proposta

auxiliar na produção de testes para o GraphQL gerando consultas válidas de forma automática, assim facilitando a criação de testes evitando trabalho humano que pode ser dispensável, assim deixando livre para outras atividades.

1.6 ORGANIZAÇÃO DO TEXTO

O restante do trabalho está organizado da seguinte forma: No Capítulo 2 é apresentado a revisão bibliográfica. O Capítulo 3 se encarrega em apresentar os principais trabalhos relacionados na área de testes de APIs GraphQL e REST, bem como sobre geração de consultas GraphQL. Já no Capítulo 4 é apresentado a metodologia para o desenvolvimento do trabalho. No Capítulo 5 apresenta-se o cronograma estipulado para a elaboração do projeto. Enfim, no Capítulo 6 encontram-se as considerações finais a respeito do trabalho.

2 REVISÃO BIBLIOGRÁFICA

2.1 APIS WEB

Segundo Higginbotham (4) uma API, ou interface de programação de aplicativos, é a especificação de como um software pode interagir com outro. É melhor pensado como um contrato entre o software e os desenvolvedores que o utilizam.

Conforme Brenda Jin; Shevat (2) uma API é a interface que um programa de software apresenta a outros programas, a humanos e, no caso de APIs da Web, ao mundo por meio da Internet. Ou seja, APIs disponibilizam uma interface que possibilita acessar algum utilitário que necessitamos.

As APIs têm tido cada vez um papel mais importante no mundo atual, conforme Brenda Jin; Shevat (2) as APIs permitem que as empresas desenvolvam produtos exclusivos rapidamente. Em vez de reinventar a roda, as startups são capazes de diferenciar suas ofertas de produtos enquanto aproveitam as tecnologias existentes e exploram outros ecossistemas. Ainda segundo Brenda Jin; Shevat (2) não é nenhum segredo que a web alimenta uma grande parte da inovação de novos produtos e do mercado de tecnologia hoje. Como resultado, as APIs são mais importantes do que nunca na criação de um negócio e existem muitos modelos para incorporá-las a um produto. Em alguns casos, uma API levará a lucro direto e monetização (por meio de modelos de participação nos lucros, taxas de assinatura ou taxas de uso).

2.2 GRAPHQL

O **GraphQL** (3) é uma linguagem de consultas para APIs onde o foco está em devolver para o cliente somente o que for solicitado, assim gerando economia de dados e permitindo maior flexibilidade no uso de APIs. Outra vantagem das APIs **GraphQL** (3) é que elas permitem a solicitação de vários recursos em uma única requisição, assim evitando o disparo de diversas requisições para adquirir diferentes recursos.

Outro ponto a ser destacado é que o **GraphQL** (3) não está vinculado a nenhuma linguagem ou banco de dados específico, ou seja, para ser utilizado basta a linguagem de programação oferecer suporte.

O **GraphQL** (3) foi criado em 2012 e se tornou de código aberto pelo Facebook em 2015. Em 2019, o Facebook e outros criaram a GraphQL Foundation como uma casa neutra e sem fins lucrativos para os ativos do **GraphQL** (3) e a colaboração contínua, é hospedada pela The Linux Foundation.

As APIs **GraphQL** (3) são baseadas em *types* (tipos) e *fields* (campos), onde a partir deles são implementadas funções para trabalhar com esses dados. Nas requisições o **GraphQL** (3) trabalha com dois tipos de solicitação: as *queries* (consultas) que servem para retornar dados e as *mutations* (mutações) que servem para alterar dados. Os dados em uma requisição são

retornados no formato JSON.

Exemplo de definição de tipo:

```
type Query {
  me: User
}

type User {
  id: ID
  name: String
}
```

Figura 1 – Exemplo de definição de tipo *GraphQL*.

Exemplo de consulta para o tipo definido:

```
{
  me {
    name
  }
}
```

Figura 2 – Exemplo de consulta *GraphQL*.

Exemplo de retorno para a consulta acima:

```
{
  "me": {
    "name": "Luke Skywalker"
  }
}
```

Figura 3 – Exemplo de retorno no formato JSON *GraphQL*.

Na Figura 1 descreve-se um exemplo de definição de tipo do GraphQL onde se especifica o nome do campo e o tipo do mesmo logo em seguida. Na Figura 2 pode-se ver um exemplo de consulta GraphQL válida para a definição de tipo da Figura 1. Já na figura 3, temos o exemplo de retorno da consulta da Figura 2, já no formato JSON.

2.3 TESTE BASEADO EM PROPRIEDADES

Nesta abordagem de teste são especificadas propriedades a serem atendidas por determinado software em teste, este deve obedecer tais especificações. Ou seja, são estabelecidas pré-condições a serem atendidas, e essas condições devem ser atendidas durante a execução dos casos de avaliação de software (5).

Para desenvolvedores Haskell uma biblioteca muito popular é o QuickCheck. Segundo Mista; Russo; Hughes (7) o QuickCheck exige que os desenvolvedores especifiquem as propriedades de teste que descrevem o comportamento esperado do software. Então, ele gera um

grande número de casos de teste aleatórios e relata aqueles que violam as propriedades de teste especificadas. Ainda conforme Hughes (5) o QuickCheck sempre testa uma propriedade, gerando casos de teste adequados e verificando se a propriedade é válida em cada caso.

2.4 GERAÇÃO DE CÓDIGO ALEATÓRIO

Geração de código aleatório é um procedimento explorado na área de testes, onde segundo Pałka (9) um possível método de geração pode ser obtido lendo as regras de tipo. Com regras pré-estabelecidas podem ser gerados códigos válidos para posterior realização de testes.

A geração de código aleatório é muito importante para realização de testes, conforme Pagrut et al. (8) a criação de software que é sustentável e escalável precisa de testes. No entanto, o processo de criação manual de casos de teste pode ser tedioso, e é por isso que a automação de teste é interessante. Os benefícios da automação de teste incluem menos trabalho repetitivo, redução de custos e facilidade de manutenção.

3 TRABALHOS RELACIONADOS

3.1 TESTE DE DESVIO: UMA GERAÇÃO DE CASOS DE TESTE - TÉCNICA PARA APIS GRAPHQL

Vargas et al. (11) criaram uma ferramenta para testar APIs GraphQL de forma automática explorando a abordagem de desvio, ou seja, a partir de uma consulta GraphQL informada que fosse válida, a ferramenta gera novos casos de testes, variações da consulta original. Após gerados esses novos casos, eram realizados os testes e verificado se os resultados obtidos das novas variações de casos de teste eram um subconjunto do caso de teste original, ou mesmo se continha erros.

Além de desenvolver a ferramenta Vargas et al. (11), foram realizados 3 estudos de caso para ver a capacidade da ferramenta em encontrar falhas. Ao analisar o resultado dos estudos de caso foi constatado que a ferramenta foi capaz de encontrar casos de erro no retorno das consultas.

Como principais diferenças em relação ao trabalho de Vargas et al. (11), o presente trabalho vai realizar a geração das consultas baseado em especificação, enquanto o trabalho de (11) faz a geração das consultas através de desvios de uma consulta original pré-estabelecida.

3.2 AUTOGRAPHQL: UMA FERRAMENTA DE GERAÇÃO DE TESTE AUTOMATIZADA PARA GRAPHQL

Zetterlund (12) implementou uma ferramenta para realização de teste de APIs GraphQL chamada AutoGraphQL, esta ferramenta funciona da seguinte forma: ela coleta consultas realizadas na API em produção e armazena em um banco de dados, para posteriormente utilizar esses dados como entradas para as consultas geradas baseadas no esquema da API. Como tecnologias usadas para implementação do AutoGraphQL, foi utilizado a linguagem Python e os casos de teste gerados para validação das consultas com o PHPunit, este último utilizado para validar se os dados retornados nas consultas possuíam a tipagem correta.

No trabalho de Zetterlund (12), a criação da ferramenta AutoGraphQL foi utilizada para um estudo de caso em específico, onde foram coletados dados de consultas durante 33 dias, e posteriormente efetuado os testes na API GraphQL em análise. Como resultados obtidos, eles conseguiram encontrar 157 falhas na API num total de 24.049 de casos de testes gerados.

O trabalho proposto neste projeto de pesquisa se diferencia do trabalho de Zetterlund (12) por fazer a geração de consultas de forma aleatória baseado no esquema da API, ou seja, não necessita de consultas prévias para criação dos casos de teste, bem como as tecnologias utilizadas para elaboração da ferramenta que neste trabalho será utilizado a linguagem funcional Haskell.

3.3 TESTE AUTOMÁTICO BASEADO EM PROPRIEDADE DE APIS GRAPHQL

Karlsson; Causevic; Sundmark (6) apresentam um dos primeiros trabalhos que propõe a geração de consultas GraphQL de forma aleatória para testes de API GraphQL. Uma vez criada as consultas são realizados testes na API e então verificado o retorno para apuração se a resposta da API está em conformidade com a definição de tipo especificados no esquema. No mesmo trabalho foi proposto observar a cobertura de casos de teste atingido pelas consultas geradas de forma aleatória, bem como analisar a aplicação da ferramenta desenvolvida para um estudo de caso.

Como resultado do trabalho de Karlsson; Causevic; Sundmark (6) foi observado que foi possível atingir uma grande cobertura de casos de teste, em algumas configurações até em 100%. No estudo de caso também foi possível observar que a ferramenta foi capaz de encontrar falhas na API em análise de teste, atendendo assim com seu propósito.

As principais distinções propostas no presente trabalho está na tecnologia utilizada para geração das consultas utilizando a linguagem funcional Haskell diferente da linguagem Clojure usada no trabalho de Karlsson; Causevic; Sundmark (6), bem como o foco do trabalho que é criar um gerador de consultas e utilizar o teste baseado em propriedades para analisar as consultas geradas, já no trabalho de Karlsson; Causevic; Sundmark (6) não foi adentrado em como são geradas as consultas, mas sim a averiguação da propriedade de tipos depois de já executado uma consulta na API.

3.4 FUZZING DE CAIXA BRANCA E CAIXA PRETA PARA APIS GRAPHQL

Belhadi; Zhang; Arcuri (1) desenvolveram uma ferramenta para teste de APIs GraphQL que usa duas abordagens, testes de caixa branca, quando se possui acesso ao código fonte e testes de caixa preta quando não se possui acesso ao código fonte, onde os casos de teste de caixa branca utilizam uma abordagem evolutiva e para os testes de caixa preta uma abordagem aleatória.

No trabalho foi ressaltado uma maior precisão ao se utilizar teste de caixa branca, porém os testes de caixa preta apesar de apresentarem menor exatidão, também foram capazes de encontrar falhas nas APIs em teste, bem como tem vantagem de serem aplicados quando não se possui o acesso ao código fonte.

O trabalho de Belhadi; Zhang; Arcuri (1) não teve enfoque na geração de consultas GraphQL, mas sim na execução de casos de testes na API para verificar a resposta e analisar possíveis inconformidades, isto para os casos de teste de caixa preta. Já para o casos de testes de caixa branca foi realizado uma análise sobre o próprio código fonte da API, que o trabalho aqui proposto não pretende adentrar.

3.5 QUICKREST: GERAÇÃO DE TESTE BASEADA EM PROPRIEDADE DE APIS RESTFUL DESCRITAS PELA OPENAPI

Stefan (10) apresenta uma ferramenta para automatizar testes de APIs RESTful, utilizando como base para os testes a documentação da API descrita pela OpenAPI, que a partir da descrição é realizada a geração dos casos de teste baseados em propriedades de tipos. O trabalho visa diminuir o trabalho humano para a criação dos testes e realização dos mesmos, bem como aumentar a confiabilidade das APIs RESTful onde a ferramenta desenvolvida foi aplicada.

Além de desenvolver a ferramenta QuickREST, Stefan (10) realizou um estudo de caso da aplicação da ferramenta em um caso da indústria. Com o estudo de caso foi possível constatar que a ferramenta foi capaz de encontrar erros na API testada, assim mostrando sua utilidade para testes de API.

O trabalho de Stefan (10) funciona exclusivamente para APIs RESTful, não permitindo testes para outros modelos de API, como por exemplo APIs GraphQL, que é um modelo de exposição de APIs que vem se difundindo nos últimos tempos, e será proposto no presente projeto de pesquisa.

4 METODOLOGIA

O presente trabalho se iniciou com uma análise na literatura para verificar as propostas atualmente existentes para geração de consultas GraphQL de forma aleatória. A pesquisa serviu como base para analisar as atuais proposições para tal tema e explorar ideias para o desenvolvimento do projeto. As etapas para a realização da pesquisa encontram-se descritas a seguir:

- Fase 1: *Revisão bibliográfica sobre o tema abordado.* Nessa etapa foram analisados diversos trabalhos propostos na área de geração de consultas GraphQL de forma aleatória, bem como na área de testes de APIs WEB.
- Fase 2: *Analisar o formato e especificação da linguagem.* Etapa onde se faz necessário analisar a especificação da linguagem para verificar as regras, para criação de consultas válidas que obedecem a sintaxe da ferramenta.
- Fase 3: *Criar ferramenta para geração das consultas.* Fase onde, com a especificação da linguagem GraphQL, faz-se a criação de ferramenta para a geração das consultas utilizando a linguagem de programação funcional Haskell.
- Fase 4: *Aplicação dos testes baseados em propriedade.* Com as consultas geradas deve-se aplicar os testes baseados em propriedades para verificar se as consultas geradas são válidas.
- Fase 5: *Testar as consultas geradas em uma API.* Nesta fase, deve-se realizar testes em uma API GraphQL para verificar se as consultas são interpretadas corretamente quando utilizado em um cenário de teste.

5 CRONOGRAMA

A Tabela 1 abaixo demonstra o cronograma das atividades propostas para a realização deste projeto de trabalho de conclusão de curso, onde se visa a geração de consultas válidas GraphQL de forma aleatória e automática.

Atividades	Set	Out	Nov	Dez	Fev	Mar	Abr	Mai	Jun	Jul
Revisão Bibliográfica	x	x								
Fundamentação Teórica			x							
Escrita do Projeto e Defesa				x	x					
Estudo Formato e Especificação da Linguagem						x				
Elaboração de Ferramenta							x	x		
Aplicação dos Testes Baseado em Propriedade								x	x	
Testes das Consultas em API									x	
Escrita e defesa do TCC									x	x

Tabela 1 – Cronograma previsto para as atividades propostas.

As atividades propostas neste cronograma podem sofrer leves alterações no decorrer do seu desenvolvimento conforme a necessidade.

6 CONSIDERAÇÕES FINAIS

O presente trabalho teve início com o estudo na literatura para analisar os trabalhos propostos para geração de consultas de forma automática para o GraphQL. Após realizada análise na literatura, verificou-se que pouco se existe para geração de consultas de forma aleatória para o GraphQL, viabilizando a proposta do presente trabalho que é a geração de consultas válidas de forma aleatória utilizando os testes baseados em propriedade com a linguagem de programação funcional Haskell.

Ademais, este trabalho pretende contribuir na construção de consultas de teste para a tecnologia GraphQL, gerando-as automaticamente de forma aleatória e válidas, desta forma auxiliando a comunidade de desenvolvedores, e cooperando na construção de softwares mais confiáveis e de maior qualidade.

REFERÊNCIAS

- 1 BELHADI, Asma; ZHANG, Man; ARCURI, Andrea. **White-Box and Black-Box Fuzzing for GraphQL APIs**. [S.l.]: arXiv, 2022. P. 1–22. DOI: 10.48550/ARXIV.2209.05833. Disponível em: <<https://arxiv.org/abs/2209.05833>>.
- 2 BRENDA JIN, Saurabh Sahni; SHEVAT, Amir. **Designing Web APIs**. United States of America: O'Reilly, 2018. P. 1–3.
- 3 GRAPHQL. **GraphQL**. [S.l.: s.n.]. Disponível em: <<https://graphql.org>>.
- 4 HIGGINBOTHAM, James. **Designing Great Web APIs**. United States of America: O'Reilly, 2015. P. 1.
- 5 HUGHES, John. Experiences with QuickCheck: Testing the Hard Stuff and Staying Sane. In: [s.l.: s.n.], mar. 2016. v. 9600, p. 3–10. ISBN 978-3-319-30935-4. DOI: 10.1007/978-3-319-30936-1_9.
- 6 KARLSSON, Stefan; CAUSEVIC, Adnan; SUNDMARK, Daniel. Automatic Property-based Testing of GraphQL APIs, p. 1–11, 2020.
- 7 MISTA, Agustín; RUSSO, Alejandro; HUGHES, John. Branching Processes for QuickCheck Generators. In: PROCEEDINGS of the 11th ACM SIGPLAN International Symposium on Haskell. St. Louis, MO, USA: Association for Computing Machinery, 2018. (Haskell 2018), p. 1–13. ISBN 9781450358354. DOI: 10.1145/3242744.3242747. Disponível em: <<https://doi.org/10.1145/3242744.3242747>>.
- 8 PAGRUT, Amit et al. Automated SQL Query Generator by Understanding a Natural Language Statement. **International Journal on Natural Language Computing**, v. 7, p. 01–11, jun. 2018. DOI: 10.5121/ijnlc.2018.7301.
- 9 PAŁKA, Michał H. Testing an Optimising Compiler by Generating Random Lambda Terms. In: p. 09–21.
- 10 STEFAN. QuickREST: Property-based Test Generation of OpenAPI Described RESTful APIs. In: p. 1–11. DOI: 10.1109/ICST46399.2020.00023.
- 11 VARGAS, Daniela Meneses et al. Deviation Testing: A Test Case Generation Technique for GraphQL APIs. In: p. 1–9.
- 12 ZETTERLUND, Louise. AutoGraphQL: An automated test generation tool for GraphQL, p. 1–78, jan. 2021.